

Go for Web Development

School of Electrical Engineering and Informatics, Institut Teknologi
Bandung

Why Go?

Advantages:

- **Compiled** → fast, single binary deployment
- **Statically typed** → catches bugs at compile time
- **Goroutines** → handle thousands of concurrent requests cheaply
- **Excellent stdlib** → `net/http` is production-grade
- **Simple** → small language, easy to learn

Trade-offs:

- Smaller ecosystem than Node/Python
- Less “batteries-included” than Django/Rails
- Verbosity (error handling: `if err != nil`)

The Origin Story

Go was created at Google in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson (Unix co-creator).

The problem they were solving:

- Large-scale distributed systems at Google
- Slow compilation times (C++ took minutes)
- Concurrency was hard (threads, locks, race conditions)
- Dependencies were a nightmare

Go's design goals:

1. **Fast compilation** (seconds, not minutes)
2. **Simple syntax** (easy to learn, easy to read)
3. **Built-in concurrency** (goroutines, channels)
4. **Static typing** (catch bugs at compile time)
5. **Fast execution** (compiled to machine code)

Go's Philosophy: Simplicity

Go has ~25 keywords (vs Java's ~50, C++'s ~90)

```
// Go: One way to do things  
if err != nil {  
    return err  
}
```

```
// vs Python: Multiple ways  
try:  
    ...  
except:  
    ...
```

```
// or  
if ...:  
    ...  
else:  
    ...
```

Benefits:

- Easy to learn (can learn in a weekend)
- - Code is readable (no “clever” tricks)
- - Consistent style (gofmt enforces formatting)
- - Fast onboarding (new team members productive quickly)

Go for Web: The Strengths

1. Performance

- Compiled to machine code (not interpreted)
- Single binary deployment (no runtime dependencies)
- Startup time: ~10ms (vs Node's ~100ms, Java's ~1s)

2. Concurrency

- Goroutines: lightweight threads (~2KB stack)
- Can spawn 100,000 goroutines on a laptop
- Perfect for handling many concurrent requests

3. Standard Library

- `net/http` is production-grade
- No need for frameworks (though they help)
- Batteries included (JSON, templates, crypto, etc.)

4. Type Safety

- Catches bugs at compile time
- IDE autocomplete works great
- Refactoring is safe

Go for Web: The Trade-Offs

1. Verbose Error Handling

```
// Go: Explicit error checking
result, err :=
doSomething()
if err != nil {
    return err
}
// ... use result ...

// vs Python: Exceptions
try:
    result = do_something()
except Exception as e:
    return e
```

2. Smaller Ecosystem

- Fewer libraries than Node/Python
- Some web frameworks less mature
- ORM options limited (GORM is the main one)

3. Less “Batteries-Included”

- No built-in ORM (use database/sql or GORM)
- No built-in validation library
- No built-in form handling (parse manually)

Real-World Adoption

Companies using Go for web services:

Company	Use Case
Google	Search infrastructure, YouTube backend
Uber	Geolocation service, real-time updates
Twitch	Chat system, live streaming
Dropbox	File storage service
Cloudflare	CDN, edge computing
American Express	Payment processing
Mercado Libre	E-commerce platform (Latin America)

Popular Go web frameworks

- Standard library (net/http) – most popular choice
- Gin – fast, minimal, Express-like
- Echo – lightweight, fast
- Fiber – Express-inspired, very fast

Go vs Other Languages for Web

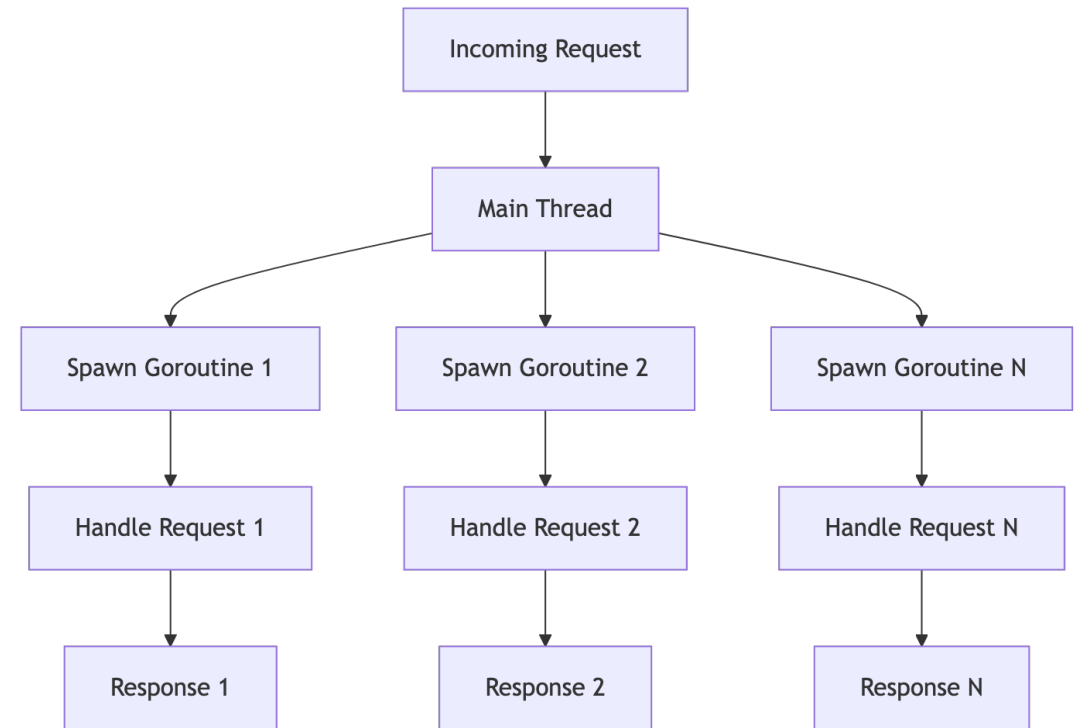
Language	Strengths	Weaknesses
Go	Fast, Safe, Concurrent	Verbose, Smaller ecosystem
Node.js	Huge ecosystem, Async	Single-threaded, Type safety
Python	Easy, Libraries	Slow, GIL limits concurrency
Java	Enterprise, Mature	Slow startup, Verbose
Rust	Fastest, Safest	Steep learning curve

When to choose Go:

- High-concurrency services (chat, real-time)
- Microservices (fast startup, small binaries)
- Performance-critical APIs
- Teams that value simplicity and type safety

Go's Execution Model

How it works: - Each request spawns a **goroutine** (lightweight thread) - Goroutines are multiplexed onto OS threads - If a goroutine blocks (I/O), others continue - Millions of goroutines possible



Execution Models

Model	How it works	Examples
Process-per-request	Spawn a process for each request	Classic PHP (Apache mod_cgi)
Thread-per-request	One thread per request	Java (Tomcat), Python (WSGI)
Event loop	Single thread, async I/O	Node.js, nginx
Goroutines (M:N)	Lightweight threads multiplexed onto OS threads	Go

A Minimal Go Web Server

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    // Define routes
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, "Hello, World!")
    })

    http.HandleFunc("/products", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, "Product list")
    })

    // Start server
    log.Println("Server starting on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

That's it. 20 lines. Production-ready. No framework needed.

Terima Kasih